

Role Assignment

Each agent has one clear responsibility.

Planner Agent

- Understand the user request
- Break it into smaller tasks
- Decide execution order

Research Agent

- Collect information
- Search documents or APIs
- Prepare research notes

Writer Agent

- Read the plan
- Read research
- Generate the final response

Coordination

The orchestrator coordinates all agents.

Orchestrator

Planner → Research → Writer

Shared Memory

- Starts agents
- Passes tasks
- Monitors progress
- Handles failures
- Collects final output

Example

Explain RAG

Planner Agent

1. Explain Retrieval
2. Explain Generation
3. Explain Benefits

Research Agent

- Embeddings
- Vector Database
- Semantic Search

Writer Agent

Output

RAG (Retrieval Augmented Generation) combines retrieval with LLMs to generate more accurate responses...

Benefits

- Specialized agents perform focused tasks
- Easier debugging
- Independent scaling
- Parallel execution is possible
- Improved overall quality

Real-World Examples

Customer Support

Agent Role

Intent Agent Identify customer intent

Knowledge Agent Retrieve relevant documents

Response Agent Generate the final reply

AI Coding Assistant

Agent Role

Planner Break coding task into subtasks

Code Agent Generate code

Reviewer Validate and improve code

Planning vs Reactive Agents

Definition

Planning Agent

Creates a plan before executing.

Reactive Agent

Responds immediately to the current situation without creating a long-term plan.

Planning Agent

A Planning Agent analyzes the goal, breaks it into smaller tasks and then executes them in sequence.

Characteristics

- Thinks before acting
- Creates a step-by-step plan
- Suitable for complex tasks
- Optimizes execution

Example

User: Build a RAG application.

Planning Agent:

1. Load documents
2. Split text into chunks
3. Generate embeddings
4. Store in vector DB
5. Build Retriever
6. Generate response

Then executes each step.

Reactive Agent

A Reactive Agent does not create a long-term plan. It observes the current input and responds immediately.

Characteristics

- Acts immediately
- No long-term planning
- Fast response
- Suitable for dynamic environments

Example

User: What is RAG?

Reactive Agent:

RAG stands for Retrieval-Augmented Generation... No planning is involved.

Comparison

Feature

Decision Making

Task Complexity

Speed

Flexibility

Memory Usage

Workflow

Planning Agent

User Request

Create Plan

Execute Steps

Reactive Agent

User Request

Immediate Response

Real-World Examples

Planning Agent

- AI Project Planner
- Travel Planner
- Multi-Agent Systems
- Software Development Assistant

Reactive Agent

- Customer support chatbot
- Weather bot
- Calculator
- Voice assistant answering simple questions

Docker

What is Docker?

Docker is an open-source containerization platform used to package an application along with all its dependencies, libraries, runtime, and configuration into a container. This ensures the application runs consistently across development, testing, and production environments.

Definition

Docker is a platform that enables developers to build, package, ship, and run applications in lightweight, portable containers.

Why Docker?

Before Docker, developers often faced the "It works on my machine" problem.

Example

A Python application works on the developer's laptop because it has:

- Python 3.11
- FastAPI
- Required libraries

When deployed to another machine:

- Python version is different
- Libraries are missing
- Environment variables are not configured

The application fails.

Docker solves this by packaging everything the application needs into one container.

Problems Solved by Docker

- Environment consistency
- Dependency management
- Faster deployment
- Application portability
- Easy scaling
- Resource efficiency

Budget

Nowhere

Your plan to build

UI React

KS

git

Backend Python

Response

Break into

Customize Support

Language has

POST

SQL

CRM

Slack

JSON

Complexity

X

Language has

POST

SQL

CRM

Slack

JSON

Complexity

X

Language has

POST

SQL

CRM

Slack

JSON

Complexity

X

Language has

POST

SQL

CRM

Slack

JSON

Complexity

X

Language has

POST

SQL

CRM

Slack

JSON

Complexity

X

Language has

POST

SQL

CRM

Slack

JSON

Complexity

X

Language has

POST

SQL

CRM

Slack

JSON

Complexity

X

Language has

POST

SQL

CRM

Slack

JSON

Complexity

X

Language has

POST

SQL

CRM

Slack

JSON

Complexity

X

AI for Dev

Retrieval

Agent Assist

Workflow

Secret Governance

Authenticated?

Session Manager

PRC Cache

Framework

Langgraph

Branch/State

Cache

Redis

VectorDB

Supabase

Python

Coding

Eval

DeepEval

API

Fast API

Scalability

Kubernetes

Deploy

Docker + AWS Cloud

Challenge

LLM → GPT-4 (Reasoning)

Cache → Redis

VectorDB → Supabase

Python → Coding

Eval → DeepEval

API → Fast API

Scalability → Kubernetes

Deploy → Docker + AWS Cloud

Can GPT-4 Answer?

Yes

No

Knowledge

No

Yes

Need first tool

Need Multi Step

Need Conversation

Workflow

ML + Python

LLM

Knowledge

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

Need Conversation

Workflow

7

re ft

The application fails.
Docker solves this by packaging everything the application needs into one container.

Problems Solved by Docker

- Environment consistency
- Dependency management
- Faster deployment
- Application portability
- Easy scaling
- Resource efficiency
- Isolation between applications
- Simplified CI/CD requirements

What is Containerization?

Containerization is the process of packaging an application together with all its dependencies into an isolated unit called a **container**.
A container includes:
• Application code
• Runtime
• Libraries
• Dependencies
• Configuration files
A container **does not** include a complete operating system.

What is a Container?

A container is a **lightweight, isolated environment** where an application runs.
Characteristics:
• Lightweight
• Fast startup
• Portable
• Isolated
• Shares the host operating system kernel
A container is a **running instance of a Docker image**.

What is a Docker Image?

A Docker image is a **read-only template** used to create containers.
It contains:
• Operating system base
• Runtime
• Libraries
• Application code
• Configuration
An image cannot run by itself.
It must be converted into a container.

Image vs Container

Image	Container
Blueprint	Running application
Read-only	Read/Write
Static	Dynamic
Stored on disk	Running in memory
Used to create containers	Created from an image

Docker Architecture

Docker consists of several components.

Docker Client

The Docker Client is the command-line interface used by developers.
Examples:
docker build
docker run
docker ps
The client sends commands to the Docker Daemon.

Docker Daemon

The Docker Daemon is the background service responsible for:
• Building images
• Running containers
• Managing networks
• Managing volumes
• Pulling images
• Removing containers
It performs all Docker operations.

Docker Engine

Docker Engine consists of:
• Docker Client
• Docker Daemon
• Docker REST API

It is the core runtime responsible for managing containers.

Docker Hub

Docker Hub is a **cloud repository** that stores Docker images.
Examples:
• python
• nginx
• ubuntu
• mysql
• redis
Developers can pull existing images instead of creating everything from scratch.

Docker Workflow

The Docker workflow consists of four major steps:
1. Write application code
2. Create a Dockerfile
3. Build an image
4. Run a container

Flow:
Application
↓
Dockerfile
↓
Docker image
↓
Docker Container

What is a Dockerfile?

A Dockerfile is a text file containing instructions for building a Docker image.
It automates the image creation process.
Common instructions:
• FROM
• WORKDIR
• COPY
• RUN
• EXPOSE
• CMD

What is Unicorn?

Unicorn is an open-source **web server** responsible for:
• Starting the application
• Listening for HTTP requests
• Passing requests to FastAPI
• Returning responses to the client
FastAPI defines APIs; Unicorn serves them.

What is Port Mapping?

Every web application listens on a network port.
Example:
Inside the container:
Application → Port 8000
Outside the container:
Browser → localhost:8000
Docker maps the host port to the container port.
Example:
docker run -p 8000:8000 image
Meaning:
Host Port 8000 → Container Port 8000

What is EXPOSE?

EXPOSE is a Dockerfile instruction used to document which port the application expects to use inside the container.
Example:
EXPOSE 8000
It does **not** publish or open the port.

COPY

COPY copies files from the local machine into the Docker image.
Example:
COPY . .

WORKDIR

sets the current working directory inside the container.
Example:
WORKDIR /app
All subsequent commands execute from this directory.

Kubernetes

1. What is Kubernetes?

Kubernetes (K8s) is an open-source **container orchestration platform** used to **deploy, manage, scale**, and maintain containerized applications.

Docker → Creates/Runs containers
Kubernetes → Manages containers at scale

2. Why Kubernetes?

Kubernetes automates:
• Deployment
• Scaling
• Load balancing
• Self-healing

3. Kubernetes Cluster

A cluster is a group of machines managed by Kubernetes.
Kubernetes Cluster

Control Plane
• Master Node
• etcd

4. Control Plane

The brain of Kubernetes. It manages the cluster.

Component	Purpose
API Server	Entry point to Kubernetes
Scheduler	Decides which node runs a Pod
Controller Manager	Maintains desired state
etcd	Stores cluster state

5. Worker Node

Component	Purpose
API Server	Entry point to <u>Kubernetes</u>
Scheduler	Decides <u>which node runs a Pod</u>
Controller Manager	Maintains <u>desired state</u>
etcd	Stores <u>cluster state</u>

A machine where applications actually run.

- Kubelet → Manages Pods ✓
- Kube-proxy → Handles networking ✓
- Container Runtime → Runs containers ✓
- Pods → Run applications

Pod is the smallest deployable unit in Kubernetes.

Pod
└ Container

Usually one application container runs inside a Pod

A Deployment manages Pods and replicas.

Deployment
↓
ReplicaSet
↓
Pods

Example: Want 3 replicas → Kubernetes maintains 3 Pods.

A Service provides a stable network endpoint for Pods

Service
↓
Pod Pod Pod

- Stable IP/DNS
- Load balancing
- Service discovery

You tell Kubernetes what you want, and Kubernetes maintains it.

Desired: 3 Pods
Actual: 2 Pods

↓
Kubernetes creates 1

Actual: 3 Beds

This provides self-healing.

3 Pods → 6 Pods → 10 Pods

- Kubernetes can increase/decrease Pods based on demand.
- Updates applications gradually without taking everything down.



